AGX Orin Bring-up and Real-Hardware Test Plan

Date	2026-10-19
Hardware	Jetson AGX Orin 64 GB (JetPack 6.x), Livox Mid-360, RoboSense Airy, ZED 2i
Designs	`docs/design/2026-10-18-dt-capture-stack-design.md`, `docs/design/2026-10-18-blender-sim-design.md`
Setup script	`scripts/jetson/setup_everything.sh`

Strategy

Each phase isolates one new source of failure. Don't start a phase until the one before it passes.

Phase	What is new	Proves	Time
A. Install	Orin software	Everything builds and the unit tests pass	3–4 h, mostly unattended
B. Sim on the Orin	Orin compute and DDS	The stack runs in real time on the Orin and reaches the laptop's accuracy	30–60 min
C. Sensors only	Real drivers, network, PTP	Data rates, time sync, IMU orientation	1–2 h
D. Live LIO + recording	Real motion	Tracking, loop return, complete sessions	1–2 h
E. Offline twin	Real data through the pipeline	Map, colour twin, change map	2–3 h
F. Camera calibration	LiDAR↔ZED extrinsic	Colour alignment	2 h (needs an x86 PC)
G. Sim vs real	Comparison	The sim predicts reality, so we can trust it	1 h

---

A. Install (unattended)

```
git clone <repo> ~/digital-twin && cd ~/digital-twin/scripts/jetson   # or copy the folder over
```

```
./00_check_system.sh                    # read it: L4T version, NVMe free
space, which NIC has PTP
./setup_everything.sh --iface eth0      # use the NIC name from 00; asks for
sudo once
```

What it does:

- **00–07:** ROS 2 Humble, Livox SDK, ZED SDK matching your L4T, GLIM, the LiDAR network, PTP, and the ROS workspace (drivers, FAST-LIO, and all `dt_*` packages including the Blender bridge).
- **08:** Python tools (`dtp`, `dt_sim`), with aarch64 checks and 25 tests.
- **09: Blender 5.2 built from source.** No official ARM64 Linux build exists anywhere: blender.org, snap, Flathub and the daily builds are all x86-only, and apt has 3.0. Blender does publish ARM64 dependency libraries, so the build compiles only Blender. This takes about 1–2 h and needs about 15 GB of disk.
- **11:** unit tests, plus the full Blender sim running on the Orin (LiDAR only).

Pass criteria:

- The summary prints every step.
- The final line is `RESULT: PASS`, with each threshold met for both days:
  - ATE < 5 cm
  - drift < 0.5 %
  - map mean error < 5 cm
  - completeness ≥ 70 %

If a step fails: fix it, then resume with `./setup_everything.sh --from NN --iface eth0`. The log is in `~/dt_data/setup_logs/`. The known failure modes are in the troubleshooting table in `scripts/jetson/README.md`.

Internet: after step 05 the RJ45 port is dedicated to the LiDAR, so use Wi-Fi for internet.

B. Simulation on the Orin

Step 11 already runs the Mid-360 sim. Also run these:

```
source ~/digital-twin/scripts/jetson/env.sh
~/digital-twin/scripts/jetson/11_verify.sh --profile airy                    #
the Airy at full rate: the DDS stress test
~/digital-twin/scripts/jetson/11_verify.sh --profile mid360 --camera         #
renders ZED frames on the Orin GPU + colour fusion
```

Checks:

1. **Mid-360 accuracy matches the laptop.** Reference from the laptop: ATE (aligned) 1.0–1.7 cm; map mean 2.6 cm; completeness 84 %. Bags are seeded, so the results should be within run-to-run noise.
2. **Airy keeps up at 1× playback on the Orin with CycloneDDS.** On the laptop with FastDDS it fell behind and only worked at 0.25×. If it fails here too, try these in order:
   - check `RMW_IMPLEMENTATION=rmw_cyclonedds_cpp`
   - check `sysctl net.core.rmem_max`
   - compose the bridge and FAST-LIO in one process
   - raise `point_filter_num`
3. **Resource use while it runs** (`sudo tegrastats` in a second terminal):

Resource	Target
CPU	under 60 %
GPU (EEVEE during `--camera`)	anything
RAM	under 16 GB

4. **EEVEE timing on the Orin.** It is logged as seconds per frame by `render_cams.py`. Expect it to be several times slower than the RTX 4060's 0.9 s per frame. If it is too slow, render cameras on the laptop and copy the run directory over (option B2).

B2. Blender on the laptop, stack on the Orin (optional)

This splits the work the way production will: the Orin only does what the real Orin will do.

```
# laptop: generate world + bags (no stack)
DT_VENV=... ./sim/run_demo.sh --profile airy --run ~/dt_data/sim/hil   # Ctrl-C
after "dt_sim: day2 bag"
rsync -a ~/dt_data/sim/hil orin:~/dt_data/sim/
# Orin: replay through the stack
~/digital-twin/sim/run_demo.sh --profile airy --run ~/dt_data/sim/hil --skip-
blender --skip-bags --no-hero
```

Watching live from the laptop:

1. On the Orin, run the bridge with `-p host:=0.0.0.0`.
2. On the laptop, open the DT Live add-on and connect to `<orin-ip>:5577`.

C. Real sensors only (no SLAM)

One-time setup:

- **Airy:** open `http://192.168.1.200` → Time Synchronization Source → **PTP-E2E (L4)** → save → power-cycle.
- **Mid-360:** IP = `192.168.1.1XX`, where XX = the last two digits of the serial. Then `export DT_MID360_IP=192.168.1.1XX`.

Launch:

```
sudo nvpmodel -m 0 && sudo jetson_clocks
ros2 launch dt_bringup capture.launch.py profile:=airy slam:=none
# then mid360
```

Pass criteria: keep the rig still and upright, and read the `dt_monitor` table.

Check	Pass	If it fails
`/dt/lidar/points` rate	9.5–10.5 Hz	Airy below 10 Hz: see the DDS fixes in B
`/dt/lidar/imu` rate	Mid-360 ~200 Hz; Airy ≥100 Hz	
LiDAR latency	0–100 ms	Huge (time since power-on): PTP not locked (`journalctl -u dt-ptp4l`). About 37 s: change phc2sys `-0`
IMU gravity	(0, 0, +9.81) ±0.3	**Airy:** fix `rotation` in `profiles/airy/bridge.yaml`. This is the main open question from research.
Camera rate	~15 Hz	ZED SDK / wrapper version mismatch: re-run 03 + 07
Camera − LiDAR latency difference	under 30 ms, stable	Note the value; it becomes `camera_time_offset`

Record 60 s of the rig sitting still, for an IMU noise and bias estimate:

```
ros2 run dt_tools dt_session record --site bench --profile airy --duration 60 --note static
```

D. Live LIO and recording (walk tests)

```
ros2 launch dt_bringup capture.launch.py profile:=airy rviz:=true
ros2 run dt_tools dt_session record --site office --profile airy --note "loop
1"
```

Protocol:

1. Stand still for 3 s.
2. Walk a closed loop of 50–100 m at a normal pace. Include one doorway or corridor and one 360° turn.
3. Stop exactly on a taped mark at the start.
4. Repeat 3× per profile.

Pass criteria:

Check	Pass
Loop-return error (`/Odometry` at end vs start)	under 0.2 m (under 0.2 % of path)
Divergence	no jumps or spinning in RViz
`dt_monitor`	odometry ~10 Hz throughout
Session folder	has `bag/` (MCAP), `zed.svo2`, `meta.yaml`, `rig.yaml`

E. Offline twin from real data

```
S=~/dt_data/sessions/office/<stamp>_airy
dtp traj $S --source fastlio && dtp map $S --voxel 0.03
ros2 run dt_tools dt_glim_config --profile airy --out
~/dt_data/glim_config/airy
ros2 run glim_ros glim_rosbag $S/bag --ros-args -p
config_path:=$HOME/dt_data/glim_config/airy
cp -r /tmp/dump $S/glim_dump && dtp traj $S --source glim && dtp map $S --voxel
0.03
dtp colorize $S                  # meaningful only after phase F
```

Checks:

- Open the map in the viewer or CloudCompare. Walls should be single, crisp surfaces with no doubling.
- The GLIM map should be at least as sharp as the FAST-LIO map at the loop closure.

- **Change test:** capture the same room twice, move 2–3 objects between captures, then:

  ```
  dtp align REF S && dtp diff REF S
  ```

  The moved objects should appear as added/removed clusters.

F. LiDAR↔camera calibration

1. `ros2 run dt_tools dt_session calib-bags --profile airy`: 6 bags. Point at feature-rich scenes and hold still (Mid-360), or sweep slowly (Airy).
2. On an x86 PC: `direct_visual_lidar_calibration` in Docker. See design §11.
3. With the ZED running: `ros2 run dt_tools dt_apply_camera_calib calib.json --profile airy`.
4. Re-run `dtp colorize` on a phase E session. Colours must line up with edges: column corners, door frames, cones.

G. Sim vs real

1. Re-run the Blender sim with the measured values:
   - the IMU noise from the static bag in C
   - the real `T_lidar_camera` from F
   - the real `camera_time_offset` from C
2. Compare against phase D/E:
   - FAST-LIO drift
   - map sharpness: point-to-plane residual on large walls
   - LiDAR point density at 5 m and 15 m
3. If the sim is much more optimistic, add the missing effects to `dt_sim`: dropout on dark or glossy materials, people walking, IMU bias.

Known open items to resolve during these tests

1. Airy IMU rotation and translation (C).
2. PTP lock on the Orin NIC for both LiDARs (C).
3. ZED v5 topic names on the installed wrapper version (C).
4. Airy 1× throughput on the Orin (B, C).
5. Blender ARM64 source build and EEVEE on Tegra (A, B). Untested. If it fails, use option B2.
6. The calibration file format from `direct_visual_lidar_calibration` (F).